

Zap

Lenguaje de Programación

Documentación Técnica del Lenguaje

Peña Villegas Diego Eduardo
Cruz Campos Pablo Isaias

Índice

1. Introducción	2
2. Paradigma y Diseño	2
2.1. Paradigma	2
2.2. Filosofía de diseño	2
2.3. Ámbito y gestión de memoria	2
3. Tipos de Datos	2
3.1. Literales	3
3.2. Reglas de promoción de tipos	3
4. Palabras Reservadas	3
4.1. Tipos de datos	3
4.2. Literales booleanos	4
4.3. Control de flujo	4
4.4. Funciones	4
5. Operadores	4
5.1. Aritméticos	4
5.2. Relacionales	5
5.3. Precedencia (menor a mayor)	5
6. Funciones Nativas	5
6.1. <code>pr</code> — Print	5
6.2. <code>rd</code> — Read	5
6.3. <code>fn + rt</code> — Funciones de usuario	6
7. Sintaxis del Lenguaje	6
7.1. Declaración de variables	6
7.2. Condicionales	6
7.3. Ciclo <code>wh</code>	6
7.4. Ciclo <code>fo</code>	6
7.5. Funciones	6
7.6. Comentarios	7
8. Ejemplos de Código	7
8.1. Conjetura de Collatz	7
8.2. N-ésimo Número Primo	7
8.3. Aproximación de Pi por Monte Carlo	8
8.4. Rutina de Kaprekar	9
8.5. Algoritmo de Euclides Extendido	10
9. Gramática Formal (BNF simplificado)	10
10. Arquitectura del Compilador	11
10.1. Fases	11
10.2. Convenciones del código generado	11

1. Introducción

Zap es un lenguaje de programación imperativo de propósito general diseñado con objetivo pedagógico. Su función es ser fuente de un compilador completo que genera código intermedio (3AC) para la arquitectura **FIS-25**. Su diseño prioriza la simplicidad sintáctica al solo utilizar palabras reservadas de 3 palabras.

El pipeline completo comprende cuatro fases: análisis léxico con Flex, análisis sintáctico con Bison generando un AST, análisis semántico con tabla de símbolos por ámbitos, y generación de código de tres direcciones conforme a la especificación FIS-25.

2. Paradigma y Diseño

2.1. Paradigma

Zap es un lenguaje **imperativo**. Los programas se expresan como secuencias de instrucciones que modifican el estado del programa, y la modularización se logra mediante la definición de funciones. No soporta orientación a objetos, programación funcional ni concurrencia.

2.2. Filosofía de diseño

El diseño de Zap responde a tres principios fundamentales:

- **Minimalismo léxico.** Las palabras reservadas son abreviaciones cortas (**wh** en lugar de **while**, **fn** en lugar de **function**), reduciendo el ruido sintáctico sin perder legibilidad.
- **Tipado estático explícito.** Todas las variables deben declararse con un tipo antes de usarse, lo que facilita la detección de errores en el analizador semántico.
- **Traducibilidad directa.** Cada construcción del lenguaje tiene una traducción natural y eficiente a código de tres direcciones FIS-25.

2.3. Ámbito y gestión de memoria

Zap utiliza un modelo de **ámbito léxico anidado**. El ámbito global contiene las declaraciones de variables y funciones definidas fuera de cualquier bloque. Cada función y cada bloque de control introduce un nuevo ámbito que permite el *shadowing* de variables externas. La gestión de memoria es estática: no existe asignación dinámica en el lenguaje fuente.

3. Tipos de Datos

Zap soporta cinco tipos de datos primitivos.

Tipo	Keyword	Descripción
int	int	Número entero con signo. Contadores, índices y aritmética entera.
float	flt	Número real de punto flotante. Cálculos de precisión.
bool	bl	Valor lógico binario. T es verdadero (1), F es falso (0).
string	str	Cadena de caracteres entre comillas dobles. Uso limitado a salida.
array	arr	Referencia a una colección de elementos.

3.1. Literales

Tipo	Ejemplos	Patrón léxico
Entero	0, 42, 1000	[0-9]+
Flotante	3.14, 0.5	[0-9]+.[0-9]+
Booleano	T, F	Tokens dedicados
Cadena	"hola mundo"	"[^"]*"

3.2. Reglas de promoción de tipos

En operaciones aritméticas entre tipos distintos, Zap aplica las siguientes promociones implícitas durante el análisis semántico:

$$\text{bool} \prec \text{int} \prec \text{float}$$

Un operando de menor jerarquía se promueve al tipo del operando de mayor jerarquía.

4. Palabras Reservadas

4.1. Tipos de datos

Keyword	Significado
int	Tipo entero
flt	Tipo flotante
str	Tipo cadena
bl	Tipo booleano
arr	Tipo arreglo

4.2. Literales booleanos

Keyword	Significado
T	Verdadero (1)
F	Falso (0)

4.3. Control de flujo

Keyword	Significado
if	Condicional principal
ei	Rama else-if (elsif)
el	Rama else
wh	Ciclo while
fo	Ciclo for

4.4. Funciones

Keyword	Significado
fn	Definición de función
rt	Retorno de valor
pr	Impresión en consola
rd	Lectura de entrada

5. Operadores

5.1. Aritméticos

Símbolo	Operación	Tipos válidos
+	Suma	int, flt
-	Resta	int, flt
*	Multiplicación	int, flt
/	División	int, flt
%	Módulo	int

5.2. Relacionales

Símbolo	Comparación
==	Igualdad
!=	Desigualdad
>	Mayor que
>=	Mayor o igual
<	Menor que
<=	Menor o igual

5.3. Precedencia (menor a mayor)

Nivel	Operadores
1	==, !=
2	>=, <=, >, <
3	+, -
4	*, /, %

Todos los operadores tienen **asociatividad por la izquierda**.

6. Funciones Nativas

6.1. pr — Print

Escribe uno o más valores en la salida estándar. Cada argumento genera una instrucción PRINT separada en el código FIS-25.

Sintaxis: `pr(expr1, expr2, ...);`

```
1 pr("El resultado es:");
2 pr(42);
```

6.2. rd — Read

Lee un valor de la entrada estándar y lo almacena en una variable.

Sintaxis: `rd(variable);`

```
1 int n;
2 rd(n);
```

6.3. `fn + rt` — Funciones de usuario

Las funciones se declaran con `fn`, reciben parámetros tipados y retornan un valor con `rt`. El valor de retorno viaja en la variable global interna `RETVAL` del código generado.

Sintaxis: `fn nombre(tipo param, ...) { ... rt expr; }`

7. Sintaxis del Lenguaje

7.1. Declaración de variables

```
1 int x;  
2 flt pi = 3.14;  
3 bl activo = T;  
4 str saludo = "hola";
```

7.2. Condicionales

```
1 if (x > 0) {  
2     pr("positivo");  
3 } ei (x == 0) {  
4     pr("cero");  
5 } el {  
6     pr("negativo");  
7 }
```

7.3. Ciclo `wh`

```
1 int i = 0;  
2 wh (i < 10) {  
3     pr(i);  
4     i = i + 1;  
5 }
```

7.4. Ciclo `fo`

```
1 fo (i = 0; i < 10; i = i + 1) {  
2     pr(i);  
3 }
```

7.5. Funciones

```
1 fn suma(int a, int b) {  
2     rt a + b;  
3 }  
4  
5 int resultado = suma(3, 4);  
6 pr(resultado);
```

7.6. Comentarios

Los comentarios de línea se inician con `#`. No existen comentarios de bloque.

```
1 # Esto es un comentario
2 int x = 5; # comentario en linea
```

8. Ejemplos de Código

8.1. Conjetura de Collatz

Dado n , aplica $n \leftarrow n/2$ si par, $n \leftarrow 3n + 1$ si impar, hasta llegar a 1. Ejemplo:
Entrada: 27 **Salida:** 111

```
1 fn collatz(int n) {
2     int pasos = 0;
3     int rem    = 0;
4     wh (n != 1) {
5         rem = n % 2;
6         if (rem == 0) {
7             n = n / 2;
8         } el {
9             n = n * 3 + 1;
10        }
11        pasos = pasos + 1;
12    }
13    rt pasos;
14 }
15
16 int num;
17 rd(num);
18 int resultado;
19 resultado = collatz(num);
20 pr("Pasos hasta llegar a 1:");
21 pr(resultado);
```

8.2. N-ésimo Número Primo

Encuentra el N -ésimo primo por división de prueba. Ejemplo: **Entrada:** 10 **Salida:** 29

```
1 fn es_primo(int n) {
2     if (n < 2) {
3         rt F;
4     }
5     int i    = 2;
6     int rem = 0;
7     wh (i * i <= n) {
8         rem = n % i;
9         if (rem == 0) {
10            rt F;
11        }
12    }
```

```

11         }
12         i = i + 1;
13     }
14     rt T;
15 }
16
17 int n;
18 rd(n);
19 int conteo = 0;
20 int candidato = 1;
21 int primo = 0;
22 wh (conteo < n) {
23     candidato = candidato + 1;
24     primo = es_primo(candidato);
25     if (primo == 1) {
26         conteo = conteo + 1;
27     }
28 }
29 pr("El N-esimo primo es:");
30 pr(candidato);

```

8.3. Aproximación de Pi por Monte Carlo

Genera coordenadas pseudoaleatorias utilizando la estrategia de “seed” y estima $\pi \approx 4 \cdot (\text{puntos dentro del círculo})/\text{total}$.

```

1  int seed = 42;
2
3  fn siguiente_rand() {
4      seed = (seed * 1103 + 12345) % 32768;
5      rt seed;
6  }
7
8  int iters;
9  rd(iters);
10
11 int dentro = 0;
12 int i = 0;
13 int rx = 0;
14 int ry = 0;
15 int x = 0;
16 int y = 0;
17 int dist2 = 0;
18
19 wh (i < iters) {
20     rx = siguiente_rand();
21     ry = siguiente_rand();
22     x = rx % 1000;
23     y = ry % 1000;
24     dist2 = x * x + y * y;
25     if (dist2 <= 1000000) {

```

```

26         dentro = dentro + 1;
27     }
28     i = i + 1;
29 }
30
31 int pi_aprox = 4000 * dentro / iters;
32 pr("Pi aproximado (multiplicado por 1000):");
33 pr(pi_aprox);

```

8.4. Rutina de Kaprekar

Para cualquier número de 4 dígitos no repetidos, la rutina converge siempre a **6174**.

Entrada: 1234 **Salida:** 3087, 8352, 6174 en 3 pasos.

```

1 fn kaprekar_paso(int n) {
2     # Extraccion de digitos con division exacta (sin residuo)
3     int d4 = n % 10;
4     int d3 = (n % 100 - d4) / 10;
5     int d2 = (n % 1000 - n % 100) / 100;
6     int d1 = (n - n % 1000) / 1000;
7     int tmp = 0;
8
9     # Burbuja de 4 elementos, ordena d1 <= d2 <= d3 <= d4
10    if (d1 > d2) { tmp = d1; d1 = d2; d2 = tmp; }
11    if (d2 > d3) { tmp = d2; d2 = d3; d3 = tmp; }
12    if (d3 > d4) { tmp = d3; d3 = d4; d4 = tmp; }
13    if (d1 > d2) { tmp = d1; d1 = d2; d2 = tmp; }
14    if (d2 > d3) { tmp = d2; d2 = d3; d3 = tmp; }
15    if (d1 > d2) { tmp = d1; d1 = d2; d2 = tmp; }
16
17    int asc = d1 * 1000 + d2 * 100 + d3 * 10 + d4;
18    int desc = d4 * 1000 + d3 * 100 + d2 * 10 + d1;
19
20    rt desc - asc;
21 }
22
23 int num;
24 rd(num);
25 int pasos = 0;
26 wh (num != 6174) {
27     num = kaprekar_paso(num);
28     pasos = pasos + 1;
29     pr(num);
30 }
31 pr("Constante de Kaprekar alcanzada en pasos:");
32 pr(pasos);

```

8.5. Algoritmo de Euclides Extendido

Calcula $\gcd(a, b)$ y los coeficientes x, y tal que $a \cdot x + b \cdot y = \gcd(a, b)$. **Entrada:** $a=35, b=15$ **Salida:** $\text{mcd}=5, x=1, y=-2$ ($35 \cdot 1 + 15 \cdot (-2) = 5$)

```

1  int coef_x = 0;
2  int coef_y = 0;
3
4  fn euclides_ext(int a, int b) {
5      if (b == 0) {
6          coef_x = 1;
7          coef_y = 0;
8          rt a;
9      }
10     int rem = a % b;
11     int mcd = euclides_ext(b, rem);
12     int xp  = coef_x;
13     int yp  = coef_y;
14     coef_x  = yp;
15     coef_y  = xp - ((a - a % b) / b) * yp;
16     rt mcd;
17 }
18
19 int a;
20 int b;
21 rd(a);
22 rd(b);
23 int resultado = euclides_ext(a, b);
24 pr("MCD:");
25 pr(resultado);
26 pr("Coeficiente x (a*x + b*y = mcd):");
27 pr(coef_x);
28 pr("Coeficiente y (a*x + b*y = mcd):");
29 pr(coef_y);

```

9. Gramática Formal (BNF simplificado)

```

program      ::= (function_def | statement)*

function_def ::= 'fn' ID '(' param_list ')' block

param_list  ::= (type ID (',' type ID)*)?

type        ::= 'int' | 'flt' | 'str' | 'bl' | 'arr'

block       ::= '{' statement* '}'

statement   ::= declaration ';'
              | assignment ';'
              | expression ';'
              | if_statement

```

```

        | 'wh' '(' expression ')' block
        | 'fo' '(' assignment ';' expression ';'
          assignment ')' block
        | 'rt' expression ';'

declaration ::= type ID ('=' expression)?

assignment ::= ID '=' expression

if_statement ::= 'if' '(' expression ')' block
              ('ei' '(' expression ')' block)*
              ('el' block)?

expression ::= INT_LIT | FLOAT_LIT | STRING_LIT | 'T' | 'F'
            | ID | ID '(' arg_list ')'
            | 'pr' '(' arg_list ')' | 'rd' '(' expression
              ')'
            | expression OP expression | '(' expression ')'
              ,

```

10. Arquitectura del Compilador

10.1. Fases

Fase	Herramienta	Descripción
Análisis léxico	Flex	Tokeniza el fuente. Ignora espacios y comentarios.
Análisis sintáctico	Bison (LALR)	Construye el AST. Detecta errores con mensajes descriptivos.
Análisis semántico	C++	Verifica tipos y declaraciones con tabla de símbolos anidada.
Generación de código	C++	Recorre el AST y emite instrucciones FIS-25 de tres direcciones.

10.2. Convenciones del código generado

- Los temporales se nombran `t0`, `t1`, `t2`, ... y se declaran con `VAR` antes de su primera asignación.
- Las etiquetas de control de flujo se nombran `L0`, `L1`, `L2`, ...
- El valor de retorno de funciones viaja en la variable global `RETVAL`.
- Los cuerpos de funciones están envueltos en `GOTO/LABEL` para que la ejecución secuencial los salte y solo sean alcanzables vía `GOSUB`.
- Los argumentos se apilan en orden inverso (right-to-left) para compatibilidad

con la pila LIFO de FIS-25.